Đề xuất dự án: Gomoku 11x11 với agent AlphaZero tự học

Người thực hiện: Nhóm 3 người — A: Khang, B: Huy, C: Hường

Lĩnh vực: Trí tuệ nhân tạo, học tăng cường, tìm kiếm cây

Trạng thái: Bản đề xuất để thảo luận

1. Tóm tắt

Dự án xây dựng một bản game Gomoku trên bàn 11x11 chơi được qua giao diện đồ họa, kèm một agent tự quyết định nước đi bằng self-play. Agent dùng mạng ResNet nhỏ làm chính sách và đánh giá vị trí, tìm kiếm cây bằng PUCT với đánh giá theo lô, tự chơi 1000 ván rồi học từ chính dữ liệu đó. Hệ thống hỗ trợ hai bộ luật: Normal (5 quân liền nhau thắng) và Renju (đen bị hạn chế 3-3, 4-4, quá 5 quân).

Agent không dùng bảng tra cứu nước đi và không hardcode chiến thuật. Mọi quyết định đều đến từ mô hình đã huấn luyện kết hợp tìm kiếm cây.

2. Bối cảnh và động lực

Gomoku là bài toán thí nghiệm thuận tiện cho AlphaZero: trạng thái là mảng rời rạc kích thước cố định, không cần xử lý ảnh hay ngôn ngữ, nên toàn bộ công sức kỹ thuật dồn vào thuật toán tìm kiếm và huấn luyện. Ở khổ 9x9 và 11x11, số nước mỗi ván tăng rõ rệt, thuật toán phải hiệu quả về tính toán mới đủ tự chơi trong phạm vi tài nguyên của nhóm.

Biến thể Renju bổ sung điều kiện khó cho agent: mỗi nước đi phải vượt qua kiểm tra luật trước khi được đánh giá, và nước bị cấm phải bị loại khỏi cây tìm kiếm. Bài toán trở thành: một agent chơi được Gomoku 11x11 trên hai bộ luật khác nhau, mà không cần bản dịch thủ công hay bảng tra cứu nước đi.

Dự án gồm hai phần: sản phẩm chơi được với agent đủ mạnh, và hệ thống đo lường được để kiểm chứng chất lượng agent từ gốc đến ngọn (mạng, tìm kiếm, tự chơi, đánh giá).

3. Mục tiêu

3.1 Mục tiêu sản phẩm

#	Mục tiêu	Tiêu chí nghiệm thu
M1	Game Gomoku 11x11 chơi được	Người chơi đấu với máy hoặc hai máy đấu nhau qua giao diện đồ họa; hiển thị nước cuối, ván thắng, hòa
M2	Agent tự ra quyết định	Không bảng tra cứu nước đi, không heuristic hardcode; mọi nước đi đến từ mô hình + tìm kiếm cây
M3	Hai bộ luật Normal và Renju	Một kiến trúc mạng, hai bộ luật, agent chơi được cả hai
M4	Đo lường được	1000 ván đánh giá có log đầy đủ, tổng hợp được theo lượt đi trước và nguyên nhân kết thúc ván; ngân sách ván tăng được bằng cấu hình mà không phải sửa mã

3.2 Câu hỏi nghiên cứu

> **Với ngân sách 1000 ván tự chơi, agent Gomoku 11x11 đạt được trình độ nào, và win rate tăng thế nào theo từng 100 ván?**

Câu hỏi này gắn trực tiếp với mục tiêu sản phẩm và có tiêu chí đo rõ ràng. Ba hướng mở rộng sau khi đạt mục tiêu: so sánh một mô hình dùng chung cho Normal và Renju với hai mô hình riêng; đo mức tăng tốc của self-play khi gom lá từ nhiều ván song song thành một lô suy luận, so với gọi suy luận lẻ từng lá; và mở rộng ngân sách tự chơi lên 1200 hay 1500 ván để quan sát đường cong win rate tiếp tục đi thế nào.

4. Phạm vi

Trong phạm vi

- Bàn 11x11, hai bộ luật Normal và Renju.
- Lõi game tách riêng khỏi tầng AI, mọi thành phần đi qua một giao diện luật duy nhất.
- Agent AlphaZero: mạng ResNet, tìm kiếm PUCT, tự chơi, huấn luyện.
- Giao diện đồ họa cho người chơi.
- Bộ công cụ đánh giá và nhật kí thí nghiệm.

Ngoài phạm vi

- Bản 9x9, bản 13x13 hoặc bàn không vuông.
- Xử lý ảnh, nhận dạng nét viết.
- Bản phân phối cho người dùng cuối, đăng ký người chơi, cơ sở dữ liệu người chơi.
- Huấn luyện mô hình ngôn ngữ hay đa phương tiện.

5. Phương pháp

5.1 Biểu diễn trạng thái

Mỗi vị trí là tensor 4 kênh kích thước 11x11: quân của bên đang đi, quân đối phương, mask nước bị cấm theo luật, và màu quân của bên đang đi. Số kênh cố định ở cả hai bộ luật nên một kiến trúc mạng dùng chung, không cần sửa lại mô hình khi đổi luật.

5.2 Mô hình

ResNet 6 block, 64 filter. Đầu chính sách phát ra 121 giá trị (một giá trị cho mỗi ô). Đầu giá trị phát ra 1 số trong khoảng [-1, 1]. Hàm mất mát: cross entropy trên chính sách (hệ số 1.0), sai số bình phương trên giá trị (hệ số 1.0), cộng chuẩn hoá L2 trọng số (hệ số 1e-4). Tối ưu bằng Adam, tốc độ học 1e-3, giảm dần theo hàm cosin theo số bước đã huấn luyện.

5.3 Tìm kiếm cây

Tìm kiếm theo PUCT với hệ số khám phá c_puct = 1.5, chạy 200 lượt mô phỏng cho mỗi nước đi. Nhiễu Dirichlet (alpha = 0.15, epsilon = 0.25) áp dụng ở gốc cây. Nhiệt độ = 1 trong 15 nước đầu, sau đó giảm dần về 0 để chọn nước tốt nhất.

Cách đánh giá vị trí đúng theo AlphaZero: mỗi lượt mô phỏng đi từ gốc xuống một lá, lá đó được đánh giá bằng đúng một lượt suy luận mạng. Lượt suy luận đó trả về hai thứ cùng lúc: đầu chính sách cho xác suất của cả 121 ô, dùng làm tiên đoán cho mọi nước đi có thể đi từ lá; đầu giá trị cho giá trị của chính lá đó. Giá

trị này là góc nhìn của đối thủ nên phủ định khi truyền ngược lên gốc. Như vậy một lượt suy luận phục vụ toàn bộ nhánh con của lá, không cần suy luận riêng cho từng nước đi.

Về hiệu năng: để tận dụng GPU, các lượt suy luận được gom lại thành lô thật. Cụ thể là gom nhiều lá đang chờ đánh giá, đến từ nhiều ván self-play chạy song song, rồi gửi một lệnh suy luận duy nhất. Đây là lô ở cấp chương trình, không phải lô ở cấp cây: không gom các nước đi con của một node để suy luận riêng cho từng con, vì cách đó tốn gấp nhiều lần số lượt tính toán mạng và không phải cách làm chuẩn của AlphaZero.

5.4 Tự chơi và huấn luyện

Ngân sách mặc định: 20 vòng, mỗi vòng 50 ván, tổng 1000 ván cho mỗi bộ luật. Ngân sách nằm trong tệp cấu hình (số vòng × số ván mỗi vòng) nên tăng lên 1200 hay 1500 ván chỉ cần đổi cấu hình, không cần sửa mã. Mỗi vòng dùng 200 bước huấn luyện với batch 128 mẫu, lấy mẫu ngẫu nhiên từ buffer tối đa 50000 mẫu. Mỗi nước đi lưu lại trạng thái, phân phối xác suất mục tiêu từ cây tìm kiếm, và giá trị z tính sau khi ván kết thúc theo góc nhìn người đang đi. Dữ liệu được tăng cường bằng 8 phép đối xứng của bàn cờ, áp dụng đồng thời lên trạng thái và chính sách. Buffer tách riêng cho từng bộ luật, không trộn dữ liệu Normal với Renju. Sau mỗi vòng, mô hình mới được đấu với mô hình tốt nhất đang có, và chỉ thay thế mô hình tốt nhất khi tỉ lệ thắng đạt từ 55% trở lên.

5.5 Siêu tham số

Nhóm	Tham số	Giá trị
Tìm kiếm	số lượt mô phỏng mỗi nước	200
Tìm kiếm	hệ số khám phá c_puct	1.5
Tìm kiếm	nhiễu Dirichlet alpha / epsilon	0.15 / 0.25
Tìm kiếm	số nước dùng nhiệt độ = 1	15
Mô hình	số block / số filter	6 / 64
Mô hình	tối ưu hoá	Adam
Mô hình	tốc độ học	1e-3, giảm dần theo cosin
Mô hình	hệ số chuẩn hoá L2	1e-4
Mô hình	trọng số mất mát chính sách / giá trị	1.0 / 1.0
Huấn luyện	số bước mỗi vòng	200
Huấn luyện	batch size	128
Huấn luyện	dung lượng buffer (trước tăng cường)	50000
Ngân sách	số vòng × số ván mỗi vòng	20 × 50 = 1000 ván mỗi bộ luật
Đánh giá	ngưỡng thay thế mô hình tốt nhất	tỉ lệ thắng ≥ 55%
Đánh giá	số ván đánh giá	1000, xen kẽ lượt đầu

5.6 Nhật kí và đánh giá

Mọi ván đều ghi vào tệp JSONL, mỗi ván một dòng, gồm: mã ván, bộ luật, vòng huấn luyện, bên đi trước, bên đi sau, người thắng, nguyên nhân kết thúc ván (thắng 5 quân / bị cấm nước / hòa / hết giờ), số nước, thời gian, số lượt gọi mạng. Ghi từng nước (giá trị ước lượng trước nước đi, vài xác suất hành động cao nhất, số lượt mô phỏng) là tùy chọn, bật khi chạy thí nghiệm cần phân tích.

Bộ đánh giá chạy 1000 ván giữa agent và đối thủ (ngẫu nhiên, MCTS thuần, hoặc mô hình cũ), xen kẽ lượt đầu: ván chẵn bên A đi trước, ván lẻ bên B đi trước, cho đúng 500 ván mỗi bên. Tổng hợp lưu thành summary.json: tỉ lệ thắng khi đi trước, tỉ lệ thắng khi đi sau, tổng thắng/hòa/thua, phân bố nguyên nhân kết thúc ván. Số ván đánh giá là tham số cấu hình, mặc định 1000.

6. Thiết kế thí nghiệm

Nhóm thí nghiệm	Nội dung	Số ván	Đầu ra
E1	Agent đấu ngẫu nhiên	100	win rate, kiểm tra sàn chất lượng
E2	Agent đấu MCTS thuần, cùng số lượt mô phỏng 200 mỗi nước để so sánh công bằng	200	so sánh với tìm kiếm không có mạng
E3	Sau mỗi 100 ván tự chơi, mô hình mới đấu mô hình trước đó, xen kẽ lượt đầu	100 ván cho mỗi mốc, tổng 1000	đường cong win rate theo từng 100 ván, tách theo đi trước/đi sau
E4	Phân tích nguyên nhân thua	theo E3	phân bố nguyên nhân kết thúc ván
E5	Normal so với Renju	1000 mỗi bộ	tác động của luật hạn chế lên trình độ

Số ván của mỗi nhóm thí nghiệm là tham số cấu hình, nên có thể tăng gấp đôi để giảm sai số thống kê mà không sửa mã.

Chỉ số theo dõi: tỉ lệ thắng, tỉ lệ thắng theo lượt đi trước, tỉ lệ thắng theo màu quân, phân bố nguyên nhân kết thúc ván, giá trị mất mát trong huấn luyện, số lượt gọi mạng mỗi ván.

Về lợi thế lượt đầu: trong Gomoku, người đi trước thường thắng nhiều hơn. Báo cáo cần nêu điều này trước khi trình bày số liệu, để tỉ lệ thắng cao khi đi trước không bị hiểu nhầm là agent mạnh hơn đối thủ.

7. Kế hoạch triển khai

7.1 Các giai đoạn

Giai đoạn	Nội dung	Phụ trách
0	Khung dự án, dependency, cấu hình, kiểm thử tự động	C
1	Lõi game: bàn cờ, luật Normal, đặc trưng, người chơi, chạy ván	A
2	MCTS thuần làm đối thủ tham chiếu	A
3	AlphaZero: mạng, tìm kiếm, buffer, tự chơi, huấn luyện	B
4	Bộ đánh giá và cổng nâng cấp mô hình	C
5	Hạ tầng chạy dài, resume, chạy song song	C
6	Nhật kí thí nghiệm	B
7	Huấn luyện 11x11 đầy đủ 1000 ván	B
8	Đánh giá 1000 ván và tài liệu nguồn	C (A, B cấp dòng nguồn module mình)
9	Giao diện đồ họa	A
10	Luật Renju và tích hợp	A dẫn dắt; B sửa mask, C đổi cấu hình
11	Báo cáo tổng kết: phương pháp, kết quả thực nghiệm, phân tích nguyên nhân thua	C

Các giai đoạn được gán mốc thời gian sau khi chốt hạn nộp với giảng viên.

7.2 Phân chia công việc

Ba người, mỗi người một nhánh code, sở hữu file của nhánh mình: **A = Khang, B = Huy, C = Hường.**

A (Khang) — Lõi game, luật, người chơi, giao diện. 8 task (T3–T7, T23–T25), cộng phần dòng nguồn cho module của mình trong giai đoạn 8.

- **Mục tiêu:** bàn cờ và luật chơi đúng, người chơi cắm được vào bộ chạy ván, game chơi được bằng chuột.
- **Task và file sở hữu:** T3 `gomoku/board.py`; T4 `gomoku/rules/base.py` + `normal.py`; T5 `gomoku/features.py`; T6 `players/base.py|random.py|human.py` + `gomoku/runner.py` + `scripts/play.py`; T7 `players/mcts_pure.py`; T23 `ui/pygame_app.py`; T24 `gomoku/rules/renju.py`; T25 phần luật và `gomoku/features.py`.
- **Sản phẩm bàn giao:** `pytest` pass toàn bộ test luật, gồm 3-3, 4-4, quá 5 quân và ngoại lệ đúng 5 quân; `python scripts/play.py` chơi hết một ván với người chơi ngẫu nhiên; MCTS thuần thắng người chơi ngẫu nhiên trên 60%; bấm chuột đánh được và ván kết thúc hiện kết quả.
- **Bàn giao cho:** B nhận giao diện luật và đặc trưng ở T4–T5; C nhận bộ chạy ván và các người chơi để dựng bộ đánh giá.
- **Không đụng:** `az/`, `scripts/train.py`, `scripts/simulate.py`, `configs/`.

B (Huy) — Tầng AlphaZero. 7 task (T8–T12, T19, T20), cộng phần dòng nguồn cho module của mình trong giai đoạn 8.

- **Mục tiêu:** agent tự học từ dữ liệu tự chơi, không heuristic hardcode.

- **Task và file sở hữu:** T8 `az/model.py`; T9 `az/mcts.py`; T10 `az/replay_buffer.py`; T11 `az/selfplay.py`; T12 `az/trainer.py`; T19 `az/game_log.py`; T20 chạy train theo cấu hình (`checkpoints/`, `logs/`); T25 sửa `az/mcts.py` + `az/selfplay.py` theo mask nước cấm.
- **Sản phẩm bàn giao:** loss giảm qua 2 vòng chạy thử; 20 vòng × 50 ván chạy xong, có checkpoint và log JSONL đầy đủ; agent thắng người chơi ngẫu nhiên; test dấu giá trị trong truyền ngược pass.
- **Bàn giao cho:** C nhận checkpoint và log để chạy đánh giá 1000 ván và viết báo cáo; A nhận schema `az/game_log.py` để gắn vào `gomoku/runner.py`.
- **Không đụng:** `gomoku/rules/`, `ui/`, `scripts/arena.py`, `scripts/simulate.py`.

C (Hường) — Hạ tầng, đánh giá, tài liệu. 11 task (T1, T2, T13–T18, T21, T22, T26).

- **Mục tiêu:** ba nhánh chạy trên cùng một khung, và mọi con số cuối cùng đo được, tra được.
- **Task và file sở hữu:** T1 khung dự án; T2 `config.py` + `configs/`; T13 `az/evaluator.py` + `scripts/arena.py`; T14 `scripts/train.py` (thiết bị, ngân sách thời gian, resume); T15 `colab/`; T16 `scripts/sync_drive.py`; T17 `scripts/selfplay_shard.py`; T18 `scripts/merge_shards.py`; T21 `scripts/simulate.py`; T22 ráp `docs/provenance.md` và viết `docs/README.md`; T26 `docs/report.md`.
- **Sản phẩm bàn giao:** `pytest` chạy và CUDA available ngay từ T1; cổng nâng cấp mô hình đúng ngưỡng 55%; `train.py --resume` nối tiếp được sau khi phiên chạy bị ngắt; 1000 ván đánh giá đúng 500 ván mỗi bên đi trước kèm `summary.json`; báo cáo có đủ số, mọi số tra được từ log.
- **Bàn giao cho:** A và B nhận khung repo, cấu hình và bộ đánh giá; giảng viên nhận báo cáo.
- **Không đụng:** `az/model.py`, `az/mcts.py`, `az/trainer.py`, `gomoku/rules/`, `ui/`.

Khối lượng: Khang (A) 8 task, Huy (B) 7 task, Hường (C) 11 task. Trong đó T20 và T21 là thời gian máy chạy, không phải thời gian viết code. T22 chia: A và B viết dòng nguồn cho module của mình, C ráp thành tệp và viết hướng dẫn cài đặt.

Mốc chặn giữa các nhánh

Mốc	Xong khi	Mở khoá cho
M1	C xong T1–T2: `pytest` chạy, cấu hình nạp được	A và B bắt đầu song song
M2	A xong T4–T5: giao diện luật và đặc trưng có test	B bắt đầu T8–T9
M3	B xong T19: schema nhật kí ván chốt	A gắn vào `gomoku/runner.py`, C gắn vào `az/evaluator.py`
M4	A xong T24: `gomoku/rules/renju.py` qua test	B sửa mask nước cấm trong tìm kiếm và tự chơi

Quy ước phối hợp:

- Mỗi người sở hữu các file của nhánh mình. Task đụng nhiều file do người sở hữu file dẫn dắt, người khác gửi yêu cầu thay đổi.
- A và B đóng góp số liệu và log ván cho C viết báo cáo.

8. Tài nguyên và rủi ro

Tài nguyên: môi trường huấn luyện chạy trên máy cá nhân có GPU, không phụ thuộc dịch vụ trả phí. Chi phí phần mềm bằng không: mọi thư viện dùng đều mã nguồn mở.

Rủi ro chính

Rủi ro	Mức độ	Xử lý
Sai dấu giá trị khi truyền ngược trong tìm kiếm	Cao	Kiểm thử riêng cho bước truyền ngược, kiểm tra dấu ở mọi vòng lặp
Tìm kiếm chậm, không kịp 1000 ván	Cao	Đánh giá theo lô, đo số lượt gọi mạng mỗi ván, giảm số lượt mô phỏng nếu cần
Luật Renju tính nước cấm tốn thời gian	Trung bình	Tính một lần mỗi nước rồi lưu kết quả theo vị trí, vô hiệu hóa khi có nước mới
11x11 khó, agent học chậm	Trung bình	Chạy thử 2 vòng trước khi chạy dài, theo dõi loss và win rate từng vòng
Lợi thế lượt đầu làm sai lệch đánh giá	Trung bình	Đánh giá luôn xen kẽ lượt đầu, báo cáo tách riêng hai chỉ số
Buffer quá lớn khiến dữ liệu cũ chi phối	Thấp	Giới hạn 50000 mẫu trước khi tăng cường
Máy cá nhân không chạy đủ lâu	Thấp	Lưu checkpoint sau mỗi vòng, nối tiếp được từ checkpoint

9. Tài liệu tham khảo

1. Silver, D. và cộng sự (2017). Mastering the game of Go without human knowledge. *Nature*, 550, 354-359.
2. Silver, D. và cộng sự (2018). Mastering the game of Chess, Shogi and Go through self-play. *Nature*, 537, 433-446.
3. Schrittwieser, J. và cộng sự (2020). Mastering Atari, Chess, Shogi and Go by planning with a learned model. *Nature*, 588, 683-689.
4. Ye, H. và cộng sự (2021). Mastering Atari games with a minimalist reinforcement learning framework. *Nature*, 596, 301-307.
5. Hoang, D. và cộng sự (2018). Is AlphaZero a general solution to game AI? *arXiv:1712.01815*.
6. Luật Renju: bộ luật của IFAA, và mục từ "Renju" trong Wikipedia.
7. Quan hệ đối xứng trong bài toán bàn cờ: Bussgang, J. và cộng sự (2020). The surprising creativity of digital evolution.

10. Phụ lục A: Truy xuất nguồn mã nguồn

Dự án lập bảng truy xuất cho từng module: đường dẫn, nguồn gốc (tự viết theo đặc tả hay lấy từ tài liệu hay kho mã nguồn), mục đích sử dụng, giấy phép. Bảng này hoàn thiện trong giai đoạn 8 và được kiểm thử tự động để không sót module nào.

Thành phần	Nguồn	Dùng để làm gì	Giấy phép
`gomoku/board.py`	Tự viết theo đặc tả	Trạng thái bàn cờ 11x11, đặt/undo nước	Nội bộ
`gomoku/rules/`	Tự viết theo đặc tả	Giao diện luật, luật Normal, luật Renju theo bộ luật IFAA	Nội bộ
`gomoku/features.py`	Tự viết theo đặc tả	Mã hóa trạng thái thành 4 kênh, 8 phép đối xứng	Nội bộ
`az/model.py`	Tự viết theo kiến trúc ResNet chuẩn	Mạng chính sách và giá trị	Nội bộ
`az/mcts.py`	Tự viết theo thuật toán PUCT của AlphaZero	Tìm kiếm cây, đánh giá theo lô, nhiễu Dirichlet	Nội bộ
`az/selfplay.py`, `az/replay_buffer.py`, `az/trainer.py`	Tự viết theo đặc tả	Sinh ván, lưu dữ liệu, huấn luyện	Nội bộ
`az/evaluator.py`	Tự viết theo đặc tả	Đánh giá mô hình, cổng nâng cấp	Nội bộ
`players/`	Tự viết theo đặc tả	Người chơi ngẫu nhiên, người chơi người, MCTS thuần, agent AlphaZero	Nội bộ
`ui/`	Tự viết theo đặc tả	Giao diện đồ họa pygame	Nội bộ
`scripts/`	Tự viết theo đặc tả	Huấn luyện, chơi, đánh giá, mô phỏng	Nội bộ
Thư viện: torch, numpy, pygame, pyyaml, pytest	Gói mã nguồn mở	Học sâu, tính toán, giao diện, cấu hình, kiểm thử	Theo giấy phép từng gói

11. Phụ lục B: Sản phẩm bàn giao

Sản phẩm	Mô tả
Mã nguồn	Kho Git, có kiểm thử tự động, tài liệu hướng dẫn cài đặt và chạy
Mô hình đã huấn luyện	Checkpoint Normal và Renju, kèm log huấn luyện
Bộ công cụ đánh giá	Script đánh giá 1000 ván, tệp log ván, bảng tổng hợp
Giao diện đồ họa	Chơi được với agent, xem lại ván đã lưu
Báo cáo	Mô tả phương pháp, kết quả thực nghiệm, phân tích nguyên nhân thua